

Home > Interview > Ace Your Interview: Top 50+ Spring Boot Quest...



INTERVIEW

Ace Your Interview: Top 50+ Spring Boot Questions and Answers

By Lavish Jain

Oct 09, 2024 | 7 Min Read | 1067 Views
(Last Updated)



Are you preparing for your Spring Boot interview? Confused and scared about how to or what to prepare? Worry not, we got you covered.

This article focuses on 50+ Spring Boot questions and answers that will help you ace your interview. So, let us get started and make sure that you appear for your interview without any fear.

Table of contents

1. [Top 50+ Spring Boot Interview Q&A](#)



- What is Spring Boot?
- What are the advantages of using Spring Boot?
- What is the Spring Boot Starter?
- Explain the concept of auto-configuration in Spring Boot.
- What is the purpose of the @SpringBootApplication annotation?
- How do you create a Spring Boot application?

Top 50+ Spring Boot Interview Q&A

Spring Boot has revolutionized the way Java developers build applications by simplifying the setup and configuration process.

As it continues to gain popularity, understanding its core concepts is essential for any developer looking to excel in their career. Below is a comprehensive guide to the top 100 Spring Boot interview questions, complete with detailed answers to help you prepare effectively.

1. What is Spring Boot?

[Spring Boot](#) is an extension of the Spring framework that simplifies the process of building production-ready applications. It provides a set of conventions and defaults to reduce boilerplate code and configuration, allowing developers to focus on building their applications.

2. What are the advantages of using Spring Boot?

- Auto-configuration: Automatically configures Spring applications based on the dependencies present.
- Standalone: Can run as a standalone application with an embedded server.
- Production-ready: Comes with features like health checks, metrics, and externalized configuration.



- Microservices-friendly: Simplifies the development of microservices architectures.

3. What is the Spring Boot Starter?

Spring Boot Starters are a set of convenient dependency descriptors that you can include in your application. For example, spring-boot-starter-web includes all the necessary dependencies to build a web application.

4. Explain the concept of auto-configuration in Spring Boot.

Auto-configuration is a feature in Spring Boot that automatically configures your application based on the libraries present on the classpath. It reduces the need for manual configuration by providing sensible defaults for commonly used features.

5. What is the purpose of the @SpringBootApplication annotation?

The @SpringBootApplication annotation is a convenience annotation that combines three annotations: @Configuration, @EnableAutoConfiguration, and @ComponentScan. It is used to mark the main class of a Spring Boot application.

6. How do you create a Spring Boot application?

You can create a Spring Boot application using the [Spring Initializr](#) or by setting up a Maven or Gradle project and adding the necessary dependencies manually.

7. What is the difference between @Component, @Service, @Repository, and @Controller?

- @Component: A generic stereotype for any Spring-managed component.
- @Service: Indicates a service layer component, typically containing business logic.



- `@Repository`: Indicates a data access layer component, usually interacting with the database.
- `@Controller`: Indicates a web controller component that handles HTTP requests.

8. What is the Spring Boot Actuator?

Spring Boot Actuator is a module that provides production-ready features to help monitor and manage Spring Boot applications. It exposes various endpoints to check application health, metrics, and environment properties.

9. How do you secure a Spring Boot application?

You can secure a Spring Boot application using Spring Security. This involves configuring authentication and authorization mechanisms, defining security rules, and protecting endpoints.

10. What is the purpose of the application.properties file?

The application.properties file is used to externalize configuration in a Spring Boot application. It allows you to define various properties, such as database connection details, server port, and logging levels.

11. How do you handle exceptions in Spring Boot?

You can handle exceptions in Spring Boot using `@ControllerAdvice` and `@ExceptionHandler` annotations. This allows you to define global exception-handling logic for your application.

12. What is a Spring Boot Profile?

A Spring Boot Profile is a way to segregate parts of your application configuration and make it available only in certain environments (e.g., development, testing, production). You can define profiles in application.properties or application.yml.

13. What is the difference between @RequestMapping and @GetMapping?



- `@RequestMapping`: A more general annotation that can handle all HTTP methods.
- `@GetMapping`: A specialized version of `@RequestMapping` that is specifically for handling GET requests.

14. What is Spring Data JPA?

Spring Data JPA is a part of the Spring Data project that simplifies data access and manipulation using JPA (Java Persistence API). It provides a repository abstraction layer to reduce boilerplate code.

**Elevate Your Career Zen Class
Courses with Placement Guidance**

[Learn More](#)



15. How do you implement caching in Spring Boot?

Caching in Spring Boot can be implemented using the `@Cacheable`, `@CachePut`, and `@CacheEvict` annotations. You also need to enable caching by annotating a configuration class with `@EnableCaching`.

16. What is the role of the CommandLineRunner interface?

The `CommandLineRunner` interface allows you to execute code after the Spring Boot application has started. You can implement this interface to run specific logic at startup.

17. What is Spring Boot DevTools?

Spring Boot DevTools is a set of tools that enhances the development experience. It includes features like automatic restarts, live reload, and configurations for improved productivity.

18. How do you create a RESTful web service using Spring Boot?

To create a RESTful web service using Spring Boot, you typically:

1. Define a controller class annotated with `@RestController`.



2. Use `@RequestMapping` or its specialized annotations (e.g., `@GetMapping`, `@PostMapping`) to define endpoints.
3. Return data in JSON or XML format using `@ResponseBody`.

19. What is the purpose of the `@Value` annotation?

The `@Value` annotation is used to inject values into Spring beans from property files or environment variables. It can be applied to fields, methods, or constructor parameters.

20. How do you implement internationalization in Spring Boot?

Internationalization (i18n) in Spring Boot can be implemented using `MessageSource` and `LocaleResolver`. You define message property files for different locales and configure the application to resolve messages based on the user's locale.

21. What is the Spring Boot CLI?

The Spring Boot CLI (Command Line Interface) is a command-line tool that allows you to quickly develop Spring applications using Groovy. It provides a way to run and test applications from the command line.

22. What is a Spring Boot Starter Parent?

A Spring Boot Starter Parent is a special starter that provides default configurations for Maven projects. It simplifies dependency management by providing a set of default dependencies and plugin configurations.

23. How do you configure Spring Boot to use a database?

To configure Spring Boot to use a database, you need to:

1. Add the appropriate database dependency (e.g., H2, MySQL) to your `pom.xml` or `build.gradle`.
2. Define the database connection properties in `application.properties` or `application.yml`.



24. What is Spring Boot's embedded server?

Spring Boot includes embedded servers (like Tomcat, Jetty, or Undertow) that allow you to run your application as a standalone Java application without needing to deploy it to an external server.

25. What is the @Configuration annotation?

The @Configuration annotation indicates that a class declares one or more @Bean methods, which will be processed by the Spring container to generate bean definitions and service requests for those beans at runtime.

26. How do you create a custom Spring Boot starter?

To create a custom Spring Boot starter, you need to:

1. Create a new Maven or Gradle project.
2. Define the dependencies and configurations you want to include in your starter.
3. Package the project as a JAR and include it as a dependency in other Spring Boot applications.

27. What is the purpose of the @EnableAutoConfiguration annotation?

The @EnableAutoConfiguration annotation tells Spring Boot to automatically configure your application based on the dependencies present on the classpath. It simplifies the setup process by providing sensible defaults.

28. How do you implement logging in Spring Boot?

Spring Boot uses SLF4J with Logback as the default logging framework. You can configure logging levels in application.properties or application.yml, and you can also customize logging formats.

29. What is Spring Boot's @RestController annotation?

The @RestController annotation is a convenience annotation that combines @Controller and @ResponseBody. It is used to define a



controller that handles RESTful web services and automatically serializes responses to JSON or XML.

30. How do you test a Spring Boot application?

You can test a Spring Boot application using the Spring Test framework, which provides annotations like `@SpringBootTest`, `@MockBean`, and `@DataJpaTest` to facilitate various testing scenarios.

31. What is the purpose of the `@Autowired` annotation?

The `@Autowired` annotation is used to automatically inject dependencies into Spring beans. It can be applied to constructors, fields, or methods.

32. How do you handle validation in Spring Boot?

Validation in Spring Boot can be handled using the `@Valid` annotation in conjunction with JSR-303/JSR-380 Bean Validation API. You can also define custom validators for complex validation logic.

33. What is the difference between `@ComponentScan` and `@EnableAutoConfiguration`?

- `@ComponentScan`: Configures component scanning for Spring beans in specified packages.
- `@EnableAutoConfiguration`: Enables Spring Boot's auto-configuration feature, allowing it to automatically configure beans based on the dependencies present on the classpath.

34. What is a Spring Boot application context?

The Spring Boot application context is the central interface to the Spring IoC container. It holds the configuration and manages the lifecycle of the beans defined in the application.

35. How do you implement pagination in Spring Data JPA?



You can implement pagination in Spring Data JPA by using the Pageable interface in your repository methods. You can return a Page<T> object that contains the paginated data and metadata.

36. What is the purpose of the @Cacheable annotation?

The @Cacheable annotation is used to indicate that the result of a method should be cached. When the method is called, Spring checks if the result is already in the cache; if it is, the cached result is returned instead of executing the method.

37. What is the role of the @Transactional annotation?

The @Transactional annotation is used to define the scope of a transaction in Spring. It can be applied to methods or classes, ensuring that all operations within the scope are executed within a single transaction.

38. How do you implement a scheduled task in Spring Boot?

You can implement scheduled tasks in Spring Boot using the @Scheduled annotation. You need to enable scheduling by annotating a configuration class with @EnableScheduling.

39. What is Spring Boot's @RequestBody annotation?

The @RequestBody annotation is used to bind the HTTP request body to a Java object. It is commonly used in RESTful web services to deserialize JSON or XML data into Java objects.

40. How do you handle file uploads in Spring Boot?

You can handle file uploads in Spring Boot using the @RequestParam annotation to bind the uploaded file to a MultipartFile parameter in your controller method.

41. What is the purpose of the @ResponseBody annotation?



The `@ResponseBody` annotation indicates that the return value of a method should be serialized directly to the HTTP response body. It is commonly used in RESTful web services.

42. How do you enable CORS in a Spring Boot application?

You can enable CORS (Cross-Origin Resource Sharing) in a Spring Boot application by using the `@CrossOrigin` annotation on your controller methods or by configuring a global CORS configuration.

43. What is a Spring Boot Starter Web?

`spring-boot-starter-web` is a starter dependency that includes all the necessary components to build web applications, including Spring MVC, Jackson for JSON processing, and an embedded server.

44. How do you create a custom exception in Spring Boot?

To create a custom exception in Spring Boot, you can define a new class that extends `RuntimeException` or `Exception`. You can then use `@ControllerAdvice` to handle this exception globally.

45. What is the purpose of the `@PathVariable` annotation?

The `@PathVariable` annotation is used to bind a method parameter to a URI template variable. It is commonly used in RESTful web services to extract values from the URL.

46. How do you implement a Spring Boot REST client?

You can implement a REST client in Spring Boot using the `RestTemplate` or `WebClient` classes. These classes provide methods to make HTTP requests and handle responses.

47. What is the role of the `@ConfigurationProperties` annotation?

The `@ConfigurationProperties` annotation is used to bind external configuration properties to a Java object. It allows you to map



properties from application.properties or application.yml to a POJO.

48. What is the purpose of the @EnableCaching annotation?

The @EnableCaching annotation is used to enable Spring's annotation-driven cache management capability. It allows you to use caching annotations like @Cacheable, @CachePut, and @CacheEvict.

49. How do you implement custom logging in Spring Boot?

You can implement custom logging in Spring Boot by configuring the logging framework in application.properties or application.yml. You can also create custom loggers and appenders as needed.

50. What is the role of the @Scheduled annotation?

The @Scheduled annotation is used to indicate that a method should be executed at fixed intervals or cron expressions. It is commonly used for background tasks and scheduled jobs.

51. How do you use Spring Boot with Thymeleaf?

You can use Spring Boot with Thymeleaf by including the spring-boot-starter-thymeleaf dependency in your project. You can then create HTML templates using Thymeleaf syntax and render them from your controllers.

52. What is the purpose of the @EnableWebMvc annotation?

The @EnableWebMvc annotation is used to enable Spring MVC in a Spring application. It provides configuration for handling web requests and setting up view resolvers.

53. How do you implement a health check in Spring Boot?

You can implement a health check in Spring Boot using the Actuator module. By including the spring-boot-starter-actuator dependency,



you can access the `/actuator/health` endpoint to check the application's health status.

54. What is the role of the `@Import` annotation?

The `@Import` annotation is used to import additional configuration classes into the current Spring configuration. It allows you to modularize your configuration and reuse beans across different contexts.

55. How do you implement a custom filter in Spring Boot?

You can implement a custom filter in Spring Boot by creating a class that implements the `Filter` interface and registering it as a bean in your application context.

56. What is the purpose of the `@FeignClient` annotation?

The `@FeignClient` annotation is used to declare a Feign client for making HTTP requests to other services. It simplifies the process of creating REST clients by using declarative HTTP requests.

57. How do you enable HTTPS in a Spring Boot application?

You can enable HTTPS in a Spring Boot application by configuring an SSL certificate in your `application.properties` or `application.yml` file and setting the server port to 443.

58. What is the role of the `@Transactional` annotation in Spring Boot?

The `@Transactional` annotation is used to define the scope of a transaction in Spring Boot. It ensures that all operations within the annotated method are executed within a single transaction context.

59. How do you create a Spring Boot application with Gradle?



To create a Spring Boot application with Gradle, you need to set up a build.gradle file with the necessary dependencies and plugins. You can then use the gradle bootRun command to run the application.

60. What is the purpose of the @EnableAsync annotation?

The @EnableAsync annotation is used to enable asynchronous method execution in Spring Boot. It allows you to execute methods asynchronously using the @Async annotation.

61. How do you implement a custom converter in Spring Boot?

You can implement a custom converter in Spring Boot by creating a class that implements the Converter interface and registering it as a bean in your application context.

62. What is the role of the @ResponseStatus annotation?

The @ResponseStatus annotation is used to specify the HTTP status code that should be returned for a specific exception or method. It allows you to customize the response status for specific scenarios.

63. How do you configure Spring Boot to use Redis?

To configure Spring Boot to use Redis, you need to add the spring-boot-starter-data-redis dependency to your project and define the Redis connection properties in application.properties or application.yml.

64. What is the purpose of the @CacheEvict annotation?

The @CacheEvict annotation is used to indicate that a method should evict (remove) entries from the cache. It is commonly used when data is modified, and the cache needs to be updated.

65. How do you implement a custom error page in Spring Boot?



You can implement a custom error page in Spring Boot by creating an HTML file named `error.html` in the `src/main/resources/templates` directory. You can also define custom error-handling logic using `@ControllerAdvice`.

66. What is the role of the `@Profile` annotation?

The `@Profile` annotation is used to indicate that a bean should only be created in specific profiles. It allows you to define beans that are environment-specific.

67. How do you implement a Spring Boot application with MongoDB?

To implement a Spring Boot application with MongoDB, you need to add the `spring-boot-starter-data-mongodb` dependency and configure the MongoDB connection properties in `application.properties` or `application.yml`.

68. What is the purpose of the `@Conditional` annotation?

The `@Conditional` annotation is used to conditionally register beans based on certain conditions. It allows you to create beans that are only created if specific criteria are met.

69. How do you implement a Spring Boot application with Kafka?

To implement a Spring Boot application with Kafka, you need to add the `spring-boot-starter-kafka` dependency and configure the Kafka connection properties in `application.properties` or `application.yml`.

70. What is the role of the `@EventListener` annotation?

The `@EventListener` annotation is used to indicate that a method should listen for specific application events. It allows you to create event-driven applications in Spring Boot.

71. How do you implement a Spring Boot application with RabbitMQ?



To implement a Spring Boot application with RabbitMQ, you need to add the `spring-boot-starter`.

By preparing these spring boot questions and answers, you can ace your interview without any worry!

In case, if you want to learn more about Spring Boot and gain in-depth knowledge on full-stack development, consider enrolling for GUVI's certified [Java Full-stack Developer Course](#) that teaches you everything from scratch and make sure you master it!

Elevate Your Career Zen Class
Courses with Placement Guidance

[Learn More](#)

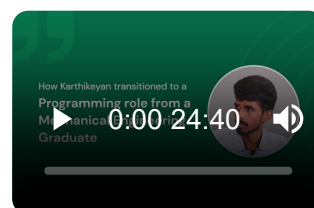
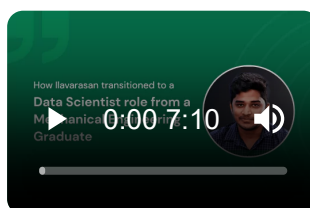
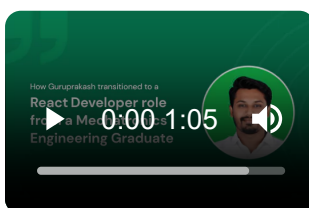


Conclusion

In conclusion, mastering Spring Boot questions and answers is essential for modern Java developers, and understanding its key concepts such as auto-configuration, dependency management, and RESTful web services will give you a significant advantage in your career. T

his guide has provided a comprehensive overview of 50+ critical Spring Boot questions and answers, covering foundational topics and advanced features like security, caching, and messaging. By thoroughly preparing with these questions, you'll be well-equipped to excel in your Spring Boot interview and confidently build robust, production-ready applications.

Career transition





About the Author

Lavish Jain

A Professional Developer Mentor and Corporate Trainer with over 10 years of experience, I have contributed to technological advancements through comprehensive training and mentorship. My journey includes pivotal roles at Cluebix, Focus-4D, ThoughtWorks, and currently with Guvi -HCL Group, where I shape the careers of aspiring developers. I specialize in simplifying complex concepts, making them accessible to learners at all levels, and have successfully trained lakhs of budding developer through corporate and boot camp courses focused on full-stack development.

Connect with me @



Did you enjoy this article?

Name

Email



Rate this Article ☆ ☆ ☆ ☆ ☆

Enter Your Comment

Submit My Comment



Learn

Full Stack Development

in your favourite programming stack

- MERN, MEAN, Java, Python & more
- Mentors from Global Companies
- Build Portfolio with 20+ Projects

Enroll Now

**Learn in english, தமிழ், ಕನ್ನಡ & हिंदी*





SOUTH ASIAN
WOMEN IN TECH

Join the World's Largest Women-Only Gen AI Learning Challenge



Sept 21st, 2024

Exclusive Access & Celebrations

Learnathon & Hackathon

Courses & Certifications

Register Now



Get In Touch For Details!

Name*

Email ID*



Phone Number*

Select your interested program*

Get In Touch Request Information

Recommended Courses

Career Programs

Micro Courses





Java Full Stack Development Course

Available in

ENGLISH TAMIL

[Know More >>](#)

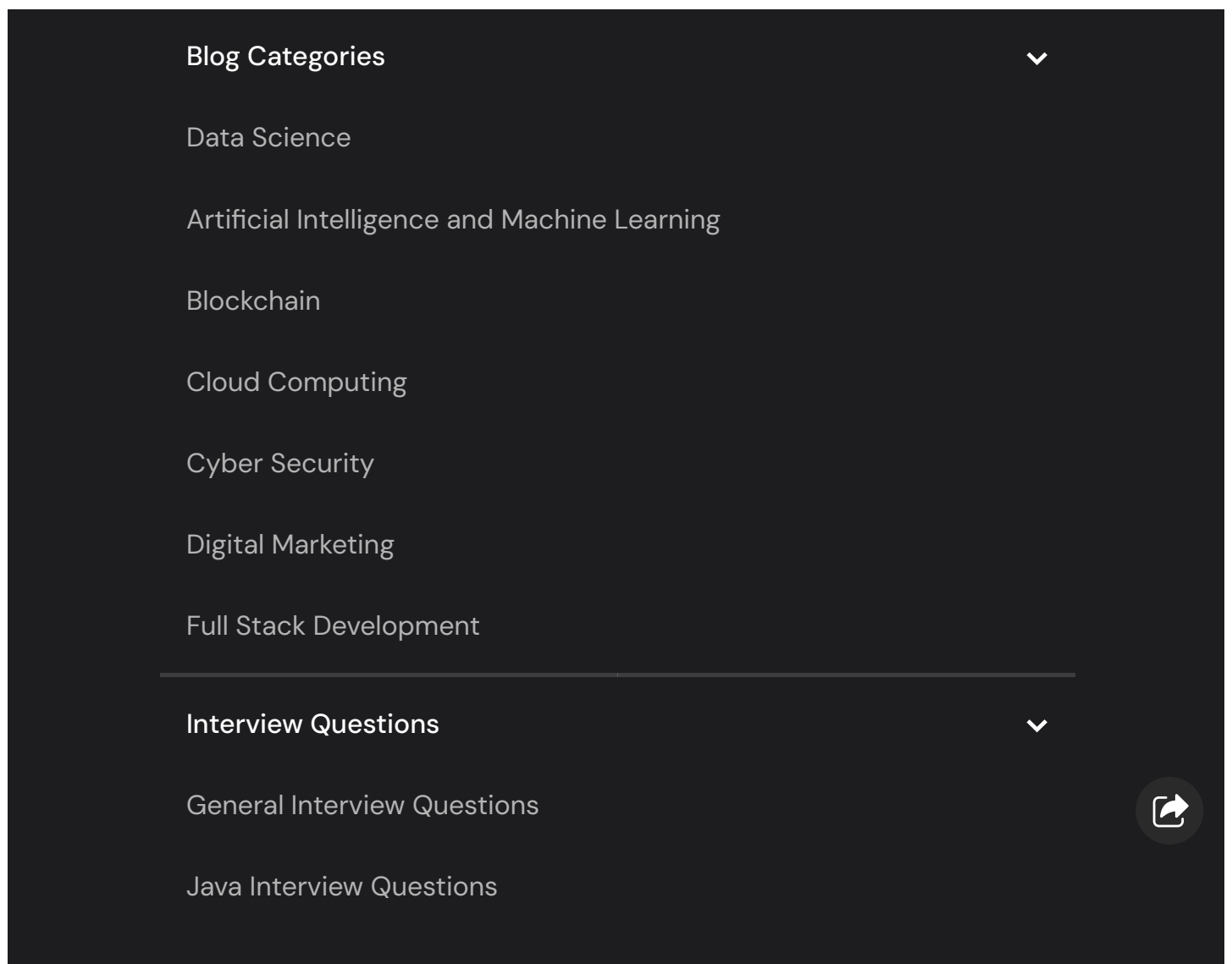
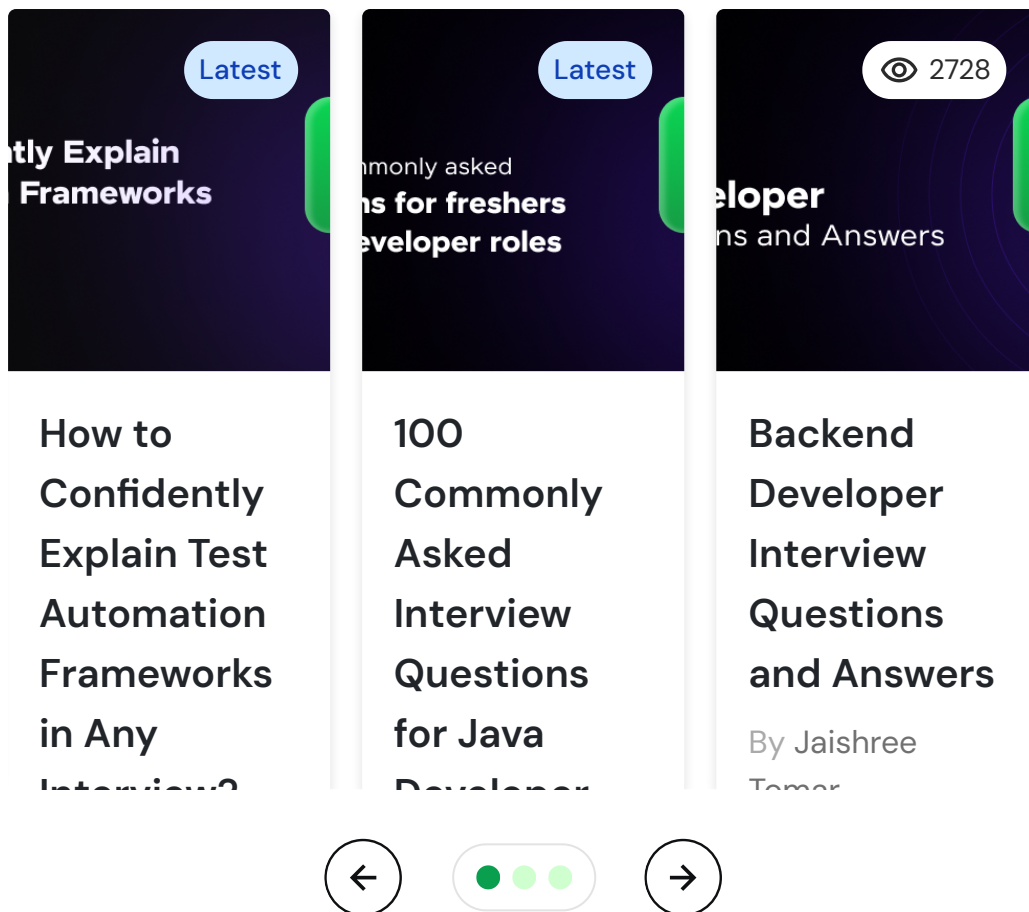


[Schedule 1:1 free counselling](#)

[Talk to Career Expert](#)

Similar Articles





[Python Interview Questions](#)

[SQL Interview Questions](#)

[Selenium Interview Questions](#)

[Data Science Interview Questions](#)

Salary blog



[UI/UX Designer Salary](#)

[Data Scientist Salary](#)

[Full-Stack Developer Salary](#)

[Motion Graphics Designer Salary](#)

[Cloud Computing Engineer Salary](#)

[Digital Marketer Salary](#)

About us



[Our Story](#)

[Careers](#)

[Refund Policy](#)

[FAQs](#)

[Contact Us](#)



[Refer & Earn](#)

GUVI – India's Pioneering Vernacular EdTech, incubated by IIT-M, IIM-A, and a prestigious part of the HCL group. Empowering over 2.5 million global learners with top-tier educational solutions through a vernacular approach, we have revolutionized global tech education with strategic partnerships, including 'Google for Education,' AICTE,



AWS, Google Cloud, IIT-M Pravartak, UiPath, and NASSCOM. GUVI is your trusted guide to inclusive and accredited learning experiences.



Follow us on



[Terms and Conditions](#)

[Privacy Policy](#)

© GUVI Geeks Network Pvt. Ltd

 [Table of contents](#)

